

Streamr

Phase 2 — Full-Stack Integration & Final Report

Project	Streamr — a Spotify-style music platform
Repositories	Spotify-FE (Next.js + TypeScript) · Spotify-BE (Django REST Framework)
Course	Web Programming, Sharif University of Technology
Team	Kiarash Sanei · Sepehr · Ava
Status	Backend complete (143 tests) · Frontend integrated (256 tests)

1. Overview

Phase 2 turns the two independently-built halves of Streamr into one working product. The backend exposes a complete JWT-secured REST API (catalog, playlists, media uploads, user preferences, reporting, and subscription payments). The frontend, previously driven entirely by a `localStorage` mock, was connected to that API through a dedicated resource layer that keeps the mock as an automatic fallback. The result runs with or without a backend, and every test on both sides passes.

2. Architecture

Backend — Spotify-BE

- Django 5.2 + Django REST Framework, camelCased over the wire (`drf-camel-case`).
- PostgreSQL 16 (the catalog uses a `Postgres ArrayField`, so SQLite is not supported).
- JWT auth (`SimpleJWT`) with access/refresh and rotation; role- and tier-based permissions.
- ASGI via Daphne with a Channels websocket consumer for group listening sessions.
- OpenAPI schema and Swagger UI via `drf-spectacular`.

Frontend — Spotify-FE

- Next.js 15 (App Router) + TypeScript, Vitest + Testing Library.
- A resource layer in `utils/resources/` maps DRF responses to the existing mock types.
- Toggle between real API and mock with a single env var, `NEXT_PUBLIC_API_URL`.
- i18n in English, Persian (RTL), and Spanish.

3. The integration approach

Rather than rewrite each screen, every data access point now calls a resource loader. When `NEXT_PUBLIC_API_URL` is set, the loader fetches from the API and maps the response into the mock shape the components already understand; when it is unset, it returns the same `localStorage` data as before. This preserved all pre-existing tests and let integration proceed screen by screen without regressions.

Resource module	Responsibility
<code>http.ts</code>	DRF pagination walker + media-URL resolver
<code>catalog.ts</code>	albums / tracks / artists -> Song / Album
<code>playlists.ts</code>	list / create / delete / detail / add-remove track

subscriptions.ts	plans + start payment (+ mock upgrade)
reports.ts	artist summary / track stats / payouts / admin overview
uploads.ts	multipart PUT for avatar / cover / audio

Wired end-to-end: authentication (login), preferences sync, listener/artist registration, catalog browsing (home, browse, album detail, recommendations, early access), playlist management, and the subscription/payment flow.

4. Payments — Zarinpal v4

The gateway integration was migrated from the retired WebGate v1 endpoints to the current Zarinpal v4 REST API. The merchant id and gateway base URL are environment-configurable, so the same code targets the sandbox in development and the live gateway in production.

- Settings → Upgrade → choose a plan.
- Frontend calls POST `/api/subscriptions/pay/start/` and receives a StartPay URL.
- The browser is sent to the Zarinpal sandbox to pay.
- Zarinpal calls the backend callback, which verifies (v4), activates the subscription, and **redirects the browser back** to `/upgrade?payment=success`.
- The frontend re-fetches `/auth/me`; the new tier (a computed property of the active subscription) appears immediately.

5. How to run

One command (needs Python 3.11+, Node 18+, PostgreSQL):

```
./run.sh # installs, writes .env, migrates + seeds, runs both test suites, serves
```

Or the whole stack in containers (ticket #43):

```
cd Spotify-BE && docker compose up --build # -> http://localhost:3000
```

Demo accounts use the password password123 (listener@, silver@, gold@, nova@ artist, support@, admin@demo.com).

6. Tickets and branches

Backend tickets already shipped on origin feature branches; two were added this phase.

Backend ticket	Branch
JWT auth / Catalog / Playlists	feat/jwt-auth, feat/catalog-crud, feat/playlists-crud
Uploads / Preferences / Reports	feat/media-uploads, feat/preferences-sync, feat/reporting-endpoints
Swagger / OpenAPI (#37)	feat/api-docs
Backend tests, clean code (#38/#41/#42) covered across the above (137 tests)	
Zarinpal v4 payments (new)	fix/zarinpal-v4-payments
Dockerize (#43, new)	feat/dockerize

Frontend integration was sliced into a reviewable stack, each branch on the previous:

Frontend ticket	Branch
1 · Resource layer	feat/api-resource-layer

2 · Catalog wiring	feat/integrate-catalog
3 · Playlists wiring	feat/integrate-playlists
4 · Registration + refreshMe	feat/integrate-registration
5 · Subscriptions / upgrade page	feat/integrate-subscriptions
6 · Dockerize (tip = full state)	feat/dockerize

7. Testing

Suite	Result
Backend (pytest)	143 passing — models, auth, CRUD, permissions, uploads, reports, payments (v4)
Frontend (vitest)	256 passing — resource mappers, mock fallback, API request/response contracts, componen

8. Status & remaining caveats

- A few dashboards (artist stats, revenue chart, accounting table) still read the mock store; their resource modules are written and tested and can be swapped in the same way the catalog was.
- Artist upload forms have a tested multipart resource but the form buttons aren't yet wired.
- The artist profile page still reads mock data (needs an `/artists/{id}/` detail loader).
- Docker Compose is YAML-validated but was not run end-to-end in the build sandbox (no Docker daemon there).
- Teammates should rebase/cherry-pick the branch tips onto the canonical main — do not force-push.

Prepared for the Streamr Phase 2 submission. Both repositories ship with full git history and the branches listed above.